

Guia de Apoio para a Apresentação (baseado no código)

Este material foi montado **somente com base no código do projeto**, para te dar segurança técnica na apresentação.

1) Resumo executivo (fala de 60–90 segundos)

"Implementei um PMV de gestão de pedidos com arquitetura de microsserviços e microfrontends. No backend, separei autenticação/usuários e pedidos em serviços FastAPI independentes, cada um com seu próprio PostgreSQL. No frontend, usei um Shell com autenticação e um MFE de pedidos carregado via Module Federation. O fluxo principal funciona de ponta a ponta: login, listagem com filtro por status, criação de pedido e atualização de status. Também incluí Redis como cache de listagem e integração opcional com IA para sugerir prioridade do pedido."

2) Arquitetura real (o que existe hoje)

Backend

- **users-service** (porta 8001)
 - Responsável por criação/listagem de usuários e login.
 - Emite JWT com **HS256**.
- **orders-service** (porta 8002)
 - Responsável por CRUD de pedidos e atualização de status.
 - Valida JWT localmente com a mesma **JWT_SECRET**.
 - Usa Redis para cache da listagem.
 - Chama IA (Anthropic) de forma opcional para prioridade/resumo.

Banco de dados

- PostgreSQL dedicado para usuários (**users-db**).
- PostgreSQL dedicado para pedidos (**orders-db**).
- Redis como cache complementar (não como fila).

Frontend

- **shell** (porta 3000): login, navegação e proteção de rotas.
 - **mfe-orders** (porta 3001): lista, criação e detalhe de pedidos.
 - Composição via Webpack Module Federation.
-

3) Como o sistema funciona (fluxo ponta a ponta)

3.1 Login

1. Usuário acessa o Shell.
2. `Login.jsx` chama `AuthContext.login()`.
3. `AuthContext` faz `POST /auth/login` no `users-service`.
4. Em sucesso, salva `token` e `user` no `localStorage`.
5. Rotas protegidas liberam acesso ao módulo de pedidos.

3.2 Listar pedidos

1. `OrdersList` chama `fetchOrders(status)`.
2. Front envia `Authorization: Bearer <token>` se houver token.
3. `orders-service` tenta ler cache (`orders:list:{status}:{skip}:{limit}`).
4. Se não houver cache, consulta PostgreSQL, serializa e cacheia por TTL.

3.3 Criar pedido

1. `OrderCreate` envia cliente + itens + notas para `POST /orders/`.
2. Backend soma `total_amount` a partir dos itens.
3. Backend chama `suggest_priority_and_summary(...)`.
4. Se houver chave da Anthropic e a chamada funcionar, grava prioridade/resumo da IA.
5. Em qualquer falha/ausência de chave, usa fallback: `priority='medium', summary=None`.
6. Invalida cache da listagem (`orders:list:*`).

3.4 Atualizar status

1. Tela de detalhe chama `PATCH /orders/{id}/status`.
2. Esse endpoint **exige token válido** (`get_current_user`).
3. Atualiza status, persiste e invalida cache da listagem.

4) Entendimento por serviço (para perguntas técnicas)

4.1 Users Service

Responsabilidades

- Cadastro de usuários.
- Login e emissão de JWT.

Modelagem

- Entidade `User`: `id`, `name`, `email`, `password_hash`, `role`, `created_at`, `updated_at`.
- `role` é enum (`admin`, `operator`).

Segurança

- Hash de senha com `passlib/bcrypt`.
- JWT emitido com expiração (`JWT_EXPIRE_MINUTES`, padrão 24h).

Endpoints-chave

- `POST /auth/login`
- `POST /users/`
- `GET /users/`
- `GET /users/{id}`
- `GET /health`

4.2 Orders Service

Responsabilidades

- Gerenciamento de pedidos.
- Filtro por status.
- Mudança de status.
- Priorização por IA (opcional).

Modelagem

- Entidade `Order`: `id`, `customer_name`, `customer_email`, `items` (JSON), `total_amount`, `status`, `priority`, `notes`, `ai_summary`, `created_by`, timestamps.
- `status`: `pending` | `processing` | `shipped` | `delivered` | `cancelled`.
- `priority`: `low` | `medium` | `high`.

Autorização

- Leitura/criação usam usuário opcional (`get_optional_user`) → podem funcionar sem token.
- Atualização de status usa usuário obrigatório (`get_current_user`) → exige token válido.

Cache Redis

- Chave de lista depende de status/paginação.
- Invalidação no create/update status.
- Degradação graciosa: se Redis cair, API segue funcionando sem cache.

IA

- Função síncrona: `suggest_priority_and_summary(customer_name, items, notes)`.
- Modelo: `claude-haiku-4-5-20251001`.
- Prompt retorna JSON com prioridade/resumo.
- Fallback robusto em erro.

5) Entendimento do frontend (o que explicar sem travar)

5.1 Shell

- Mantém estado de autenticação via `AuthContext`.
- Salva token em `localStorage`.
- Protege rota `/orders/*` com `ProtectedRoute`.
- Carrega MFE remotamente com `React.lazy(() => import('mfe_orders/OrdersApp'))`.

5.2 MFE Orders

- `OrdersList`: listagem + filtro por status.
 - `OrderCreate`: formulário com itens dinâmicos e cálculo de total no cliente.
 - `OrderDetail`: mostra resumo da IA e permite transição de status.
 - `api.js`: centraliza chamadas ao backend e injeta Bearer token quando existir.
-

6) Demonstração ao vivo (roteiro seguro)

1. Subir stack: `docker-compose up --build`.
 2. Rodar seed: `python scripts/seed.py`.
 3. Acessar `http://localhost:3000`.
 4. Login com `admin@demo.com` / `admin123`.
 5. Mostrar lista de pedidos e filtro por status.
 6. Criar novo pedido (com observação de urgência, se quiser destacar IA).
 7. Abrir detalhe e trocar status.
 8. Provar independência:
 - `docker-compose stop users-service`
 - Mostrar que listagem de pedidos ainda responde.
 - Mostrar que `PATCH /status` sem token válido falha (401).
-

7) Perguntas prováveis da banca e respostas sugeridas

"Por que FastAPI e não Django REST?"

Resposta curta:

- Menos boilerplate para PMV.
- Tipagem e validação com Pydantic simplificam.
- Swagger nativo acelera entrega.

"Como os serviços se autenticam sem gateway?"

Resposta curta:

- JWT assinado no `users-service` e validado localmente no `orders-service` com `JWT_SECRET` compartilhada.
- Evita acoplamento síncrono entre serviços para validação de token.

"Por que Redis?"

Resposta curta:

- Redução de latência e carga no PostgreSQL na listagem.
- Invalidação explícita no write mantém consistência aceitável para PMV.

"A IA é obrigatória para criar pedido?"

Resposta curta:

- Não. É opcional e resiliente.
- Sem chave ou com erro, pedido é criado com prioridade padrão.

"Onde estão os testes?"

Resposta curta:

- Cada serviço possui suíte própria com `pytest`.
- Endpoints principais estão cobertos (criação, listagem, busca, auth/status).

"Quais limites você reconhece?"

Resposta curta:

- Sem fila assíncrona.
- Observabilidade avançada não implementada (só logs estruturados).
- Sem refresh token e RBAC fino.

8) Pontos de atenção (não ser pego de surpresa)

- A regra de autorização é **assimétrica**:
 - `GET/POST /orders` aceitam usuário opcional.
 - `PATCH /orders/{id}/status` exige token.
- Testes de serviço usam SQLite de teste e mocks para Redis/IA no orders.
- Não há implementação de fila/worker.
- Não há tracing/métricas (Prometheus/OpenTelemetry).

9) Como defender escopo (1 semana, dev solo)

Fala sugerida:

"Dado o prazo e contexto de um desenvolvedor solo, priorizei garantir 100% do fluxo principal funcionando com arquitetura limpa e testável: autenticação, gestão de pedidos, cache e integração frontend-backend por microfrontend. Itens de bônus que exigem mais operação e observabilidade distribuída foram conscientemente planejados para próxima fase."

10) Checklist final antes da apresentação

- Stack sobe sem erro (`docker-compose up --build`).
- Seed executa e cria usuários/pedidos.
- Login funciona no Shell.
- Filtro por status funciona.
- Criação de pedido funciona.
- Atualização de status funciona com token.
- Endpoints `/docs` acessíveis nas portas 8001 e 8002.

- Script de slides atualizado e sem inconsistências de IA.
-

11) Arquivos de referência para estudo rápido

- Infra: `docker-compose.yml`
 - Auth/usuários: `services/users/app/main.py`, `services/users/app/auth.py`, `services/users/app/routes/auth.py`, `services/users/app/routes/users.py`
 - Pedidos: `services/orders/app/main.py`, `services/orders/app/routes/orders.py`, `services/orders/app/models.py`, `services/orders/app/schemas.py`
 - Cache/IA: `services/orders/app/redis_client.py`, `services/orders/app/ai_service.py`
 - Frontend shell: `frontend/shell/src/App.jsx`, `frontend/shell/src/context/AuthContext.jsx`
 - Frontend pedidos: `frontend/mfe-orders/src/OrdersApp.jsx`, `frontend/mfe-orders/src/components/OrdersList.jsx`, `frontend/mfe-orders/src/components/OrderCreate.jsx`, `frontend/mfe-orders/src/components/OrderDetail.jsx`, `frontend/mfe-orders/src/api.js`
 - Testes: `services/users/tests/`, `services/orders/tests/`
 - Slides: `scripts/gerar_apresentacao.py`
-

Se quiser, próximo passo eu posso criar uma **versão oral** (roteiro minuto a minuto de 20 min) para você ensaiar literalmente com frases prontas.